

As respostas estão em negrito

Para a resolução das questões pertinentes a avaliação, considere exclusivamente o esquema de BD abaixo representado pela Figura 1.

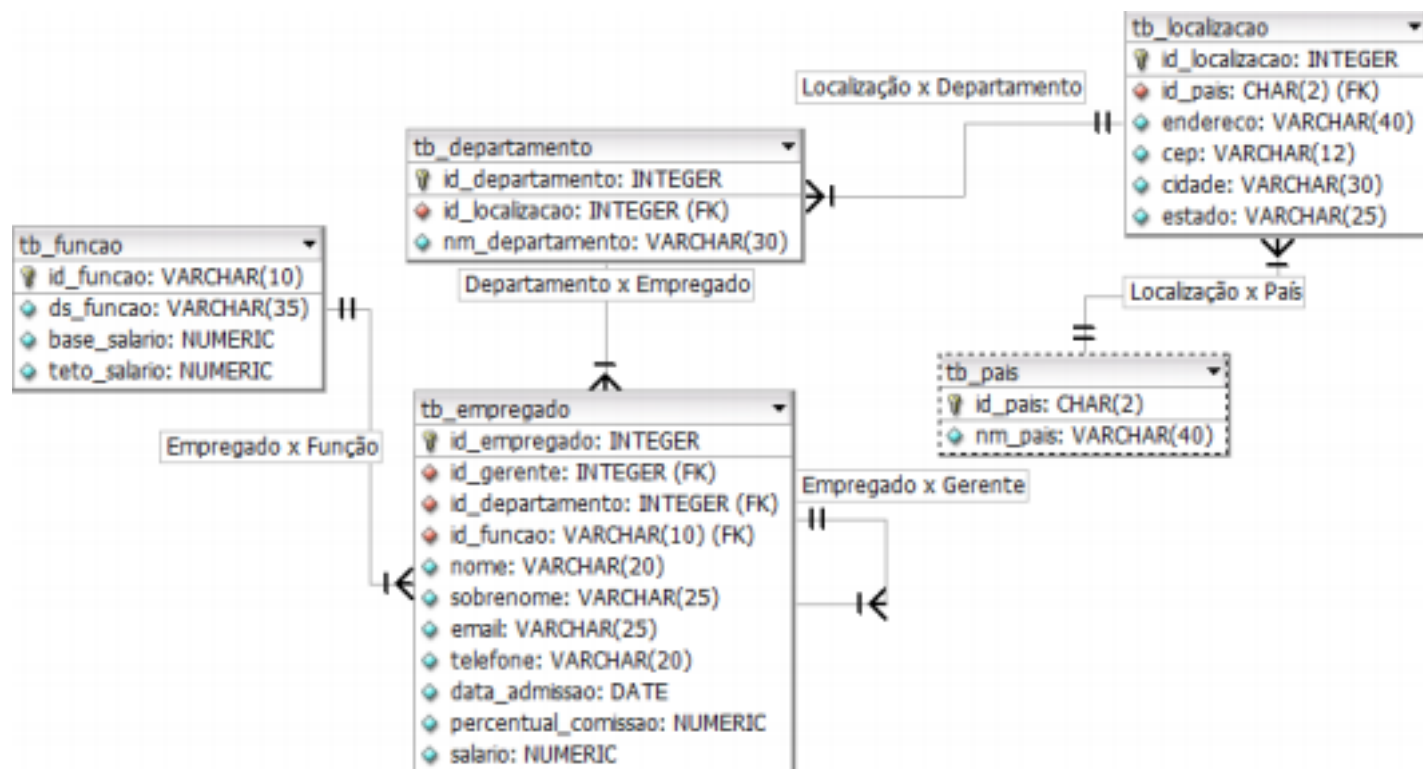


Figura 1 – Esquema de BD

1) (1,5) Utilizando a linguagem PL/pgSQL elabore uma função responsável por retornar à quantidade de empregados que desempenham determinada função, conforme as especificações a seguir:

- A função receberá como parâmetro de entrada (IN) o ID da função;
- A função deverá retornar a ID da função, DESCRIÇÃO da função e QUANTIDADE de empregados que desempenham determinada função.
- Caso ID da função informado como parâmetro não existir, exibir uma mensagem informativa.

Exemplos:

-- Invocando a função e passando como parâmetro o ID da função IT_PROG

```
fn_exerc_01
character varying
Sigla: IT_PROG - Descrição da Função: Programador - Quantidade: 2
```

-- Invocando a função e passando como parâmetro o ID da função inexistente

fn_exerc_01
character varying
Sigla da Função Inexistente

```
SELECT *  
FROM tb_empregado;
```

```
SELECT e.id_funcao, f.ds_funcao, COUNT(1) total  
FROM tb_empregado e  
INNER JOIN tb_funcao f on (e.id_funcao = f.id_funcao)  
GROUP BY e.id_funcao, f.ds_funcao;
```

```
CREATE OR REPLACE FUNCTION fn_exerc_01(p_sigla VARCHAR)  
RETURNS varchar AS  
$$  
DECLARE  
v_registro RECORD;  
v_msg VARCHAR;  
BEGIN  
SELECT e.id_funcao, f.ds_funcao, COUNT(1) total INTO v_registro  
FROM tb_empregado e  
INNER JOIN tb_funcao f on (e.id_funcao = f.id_funcao)  
GROUP BY e.id_funcao, f.ds_funcao;  
  
IF (v_registro.id_funcao IS NULL) THEN  
v_msg := 'Sigla da função inexistente';  
ELSE  
v_msg := 'Sigla: ' || v_registro.id_funcao ||  
        ' - Descrição da Função: ' || v_registro.ds_funcao ||  
        ' - Quantidade: ' || v_registro.total;  
END IF;  
  
RETURN v_msg;  
END;  
$$  
LANGUAGE 'plpgsql';
```

```
SELECT fn_exerc_01('IT');
```

2) (1,5) Utilizando a linguagem PL/pgSQL elabore uma função responsável por exibir o nome completo do empregado, nome do departamento e nome do país onde o departamento se encontra localizado.

- A função deverá aceitar um parâmetro de entrada (modo IN) que especificará o ID do empregado o qual deverá ser apresentado os dados;
- Caso ID do empregado informado como parâmetro não existir, exibir uma mensagem informativa;
- Utilize, obrigatoriamente, um registro para manipular as variáveis a serem exibidas.

Exemplos:

-- Invocando a função e passando como parâmetro o ID de um empregado válido

```
fn_exerc_02  
character varying  
Nome Completo: Steven King - Nome Depto: Administração - Nome País: Itália
```

-- Invocando a função e passando como parâmetro o ID de um empregado inválido

```
fn_exerc_02  
character varying  
Empregado 999 não localizado!!!
```

```
CREATE OR REPLACE FUNCTION fn_exerc_02 (p_id_emp IN  
tb_empregado.id_empregado%Type)  
RETURNS varchar AS  
$$  
DECLARE  
v_registro RECORD;  
BEGIN  
  
SELECT e.nome || " || e.sobrenome FullName, d.nm_departamento, p.nm_pais INTO v_registro  
FROM tb_empregado e  
INNER JOIN tb_departamento d USING (id_departamento)  
INNER JOIN tb_localizacao l USING (id_localizacao)  
INNER JOIN tb_pais p USING (id_pais)  
WHERE e.id_empregado = p_id_emp;  
  
IF NOT FOUND THEN  
RETURN 'Empregado ' || p_id_emp || ' não localizado!';  
END IF;  
  
IF FOUND THEN  
RETURN 'Nome Completo: ' || v_registro.FullName ||  
    ' - Nome Depto: ' || v_registro.nm_departamento ||  
    ' - Nome País: ' || v_registro.nm_pais;  
END IF;  
END;  
$$  
LANGUAGE 'plpgsql';  
  
SELECT fn_exerc_02 (100);
```

- 3) (2,0) Utilizando a linguagem PL/pgSQL elabore uma função responsável calcular o índice de massa corpórea (IMC) de uma determinada pessoa, gerando uma mensagem resultante, contendo sua categoria, baseando-se no valor obtido pelo cálculo do IMC:



- A função deverá aceitar como parâmetros de entrada (modo IN) o nome, sobrenome, altura e peso.
- Toda vez que a função for invocada, a mesma deverá inserir uma *tupla* na tabela intitulada de “**tb_imc**”, conforme a estrutura abaixo:

```
CREATE TABLE tb_imc(
  id_imc SERIAL,
  nome VARCHAR(30),
  sobrenome VARCHAR(30),
  altura NUMERIC(5,2),
  peso NUMERIC(5,2),
  imc NUMERIC(4,2),
  CONSTRAINT pk_tb_imc_id PRIMARY KEY(id_imc));
```

- Além de inserir a *tupla* na “**tb_imc**”, a função deverá retornar uma mensagem, exibindo o nome, sobrenome, o IMC e a respectiva categoria, conforme exemplo:

-- Invocando a função e passando como parâmetro o ID de um empregado válido

```
SELECT fn_exerc_03('Ruth', 'Mendes', 212.3, 1.70);
```

```
fn_exerc_03
character varying
Olá: Ruth Mendes - IMC: 73.5 - Sua categoria: Obesidade Mórbida
```

```
CREATE OR REPLACE FUNCTION fn_exerc_03(p_nome VARCHAR,
                                         p_sobrenome tb_imc.sobrenome%TYPE,
                                         p_peso tb_imc.peso%TYPE,
                                         p_altura tb_imc.altura%TYPE )
```

```
RETURNS varchar AS
```

```
$$
```

```
DECLARE
```

```
v_categoria VARCHAR;
```

```
v_imc NUMERIC(3,1);
```

```
BEGIN
```

```
SELECT fn_exerc_03('Tarcisio', 'Ribeiro', 66.9, 1.70);
```

- A 1ª noite custa R\$ 100,00
- A 2ª noite custa R\$ 50,00 ($\text{R\$ } 100,00 / 2$)
- A n-ésima noite custa $\text{R\$ } 100,00 / n$

- ```
SELECT fn exerc 04('Geraldo Henrique', 'São Paulo', 4);
```

Nome completo: Geraldo Henrique - Cidade: São Paulo - Total à pagar: R\$208.33

```
CREATE OR REPLACE FUNCTION fn_exerc_04(p_nome VARCHAR,
```

```

RETURNS varchar AS
$$
DECLARE
 v_pagar NUMERIC (7,2) := 0.00;
BEGIN
 FOR i IN 1..p_nr_noites LOOP
 v_pagar := v_pagar + (100.00 / i);
 END LOOP;

 RETURN 'Nome Completo: ' || p_nome ||
 ' - Cidade: ' || p_cidade ||
 ' - Total à pagar R$' || v_pagar;
END;
$$
LANGUAGE 'plpgsql';

```

- 5) (1,0) Utilizando a linguagem DDL, crie adequadamente a tabela intitulada de “*tb\_empregado*”. Impreterivelmente, utilize explicitamente nomes para todas as restrições existentes na tabela.

```

CREATE TABLE tb_empregado(
 id_empregado INTEGER,
 id_gerente INTEGER,
 id_departamento INTEGER,
 id_funcao VARCHAR(10),
 nome VARCHAR(20),
 sobrenome VARCHAR(25),
 email VARCHAR(25),
 telefone VARCHAR(20),
 data_admissao DATE,
 percentual_comissao NUMERIC,
 salario NUMERIC,
 CONSTRAINT pk_tb_emp_id_emp PRIMARY KEY (id_empregado),
 CONSTRAINT fk_tb_emp_id_gerente FOREIGN KEY (id_gerente)
 REFERENCES tb_empregado (id_empregado),
 CONSTRAINT fk_tb_emp_id_depto FOREIGN KEY (id_departamento)
 REFERENCES tb_departamento (id_departamento),
 CONSTRAINT fk_tb_emp_id_funcao FOREIGN KEY (id_funcao)
 REFERENCES tb_funcao (id_funcao);

```

- 6) (0,5) Quantos registros são retornados pela seguinte consulta?

```

SELECT SUM(salary), department_id
FROM employees
GROUP BY department_id;

```

Suponha que haja 11 valores DEPARTMENT\_ID não nulos exclusivos e 11 nulos. Todos os registros têm um valor SALARY não nulo. (Escolha a melhor resposta).

- A. 10
- B. 11
- C. 12 (correta)

D. NULL

7) (0,5) Quais valores são retornados após a execução da seguinte declaração?

```
SELECT job_id, max_salary
FROM jobs
GROUP BY max_salary;
```

Suponha que a tabela JOBS tenha dez registros com o mesmo valor JOB\_ID de DBA e o mesmo valor MAX\_SALARY de 100. (Escolha a melhor resposta).

**A. Um erro é retornado (correta)**

B. Uma linha de saída com os valores DBA, 100

C. Dez linhas de saída com os valores DBA, 100

D. Dez linhas de saída com os valores DBA, 200

8) (0,5) Selecione uma instrução correta em relação à seguinte consulta.

```
SELECT * FROM EMPLOYEES E
JOIN DEPARTMENTS D
 ON (D.DEPARTMENT_ID = E.DEPARTMENT_ID)
JOIN LOCATIONS L
 ON (L.LOCATION_ID = D.LOCATION_ID);
```

A. Unir três tabelas não é permitido

B. Unir duas tabelas não é permitido

C. Um produto cartesiano é gerado

**D. A cláusula JOIN...ON pode ser usada para junções entre diversas tabelas (correta)**

9) (0,5) Quantas linhas são retornadas após executar a seguinte instrução?

```
SELECT * FROM REGIONS R1
JOIN REGIONS R2 ON (R1.REGION_ID = LENGTH(R2.REGION_NAME)/2);
```

A tabela REGIONS contém os seguintes dados da linha (Escolha a melhor resposta).

| REGION_ID | REGION_NAME            |
|-----------|------------------------|
| 1         | Europe                 |
| 2         | Americas               |
| 3         | Asia                   |
| 4         | Middle East and Africa |

- A. 1
- B. 2
- C. 3 (correta)**
- D. 4

10) (0,5) Um Analista de Sistemas realizou a seguinte instrução SQL:

```
SELECT COUNT(*)
FROM tb_produtos
WHERE (fornecedor NOT IN (1,4,9))
AND (preco BETWEEN 97 AND 450);
```

Considere a tabela *tb\_produtos*, a seguir:

| codigo<br>integer | produto<br>character varying(15) | fornecedor<br>integer | preco<br>integer |
|-------------------|----------------------------------|-----------------------|------------------|
| 1                 | Scanner                          | 1                     | 750              |
| 2                 | Teclado                          | 7                     | 95               |
| 4                 | Tonner                           | 3                     | 120              |
| 7                 | Monitor                          | 1                     | 450              |
| 9                 | Pendrive                         | 7                     | 25               |
| 14                | Mouse                            | 4                     | 32               |

Assinale a alternativa que apresenta o valor que a instrução SQL irá retornar:

- A. 0
- B. 1 (correta)**
- C. 2
- D. 3